



Cortex: A Retrieval-Augmented Framework for Career-Aligned Learning Paths

Vatsala Ramachandran¹ · Manasvi Vedanta¹ · Sonia Khetarpaul¹

Received: 15 March 2026 / Revised: 18 July 2026 / Accepted: 27 August 2026

© The Author(s), under exclusive licence to Springer Science+Business Media LLC, part of Springer Nature 2026

Abstract

In an era of rapidly evolving professional development, many learners and individuals transitioning careers find it challenging to align their existing skills with the demands of their desired professions. Conventional career guidance systems often rely on rigid keyword matching, which overlooks the semantic nuances of users' intentions. For instance, a user interested in "interpreting big data" may not explicitly mention "Data Scientist," causing traditional systems to miss this connection. Furthermore, once a career path is identified, determining an optimal sequence for skill acquisition—while considering prerequisites and dependencies—remains a significant pedagogical challenge that is rarely automated effectively at scale. To address these challenges, this paper proposes **Cortex**, an AI-based career-aligned learning system that combines retrieval-augmented generation (RAG), large language models (LLMs), knowledge graphs, and sentence embeddings to semantically match user interests with occupations and generate prerequisite-consistent upskilling paths with inter-skill dependencies and scaffolding. The primary scientific contribution is a novel end-to-end architecture connecting semantic intent understanding, prerequisite-aware skill sequencing via ESCO dependency graphs, and taxonomy-constrained LLM generation. The system leverages the ESCO dataset to map career goals to relevant skills and produce structured learning paths. Experimental results demonstrate that integrating LLMs with a constrained RAG framework and knowledge graph-based dependency modeling can effectively generate prerequisite-aware learning paths for diverse career goals.

Keywords Learning path generation · Large language models · Retrieval-augmented generation · Knowledge graphs · Career recommendation systems

Editors: Lan Du, Benjamin C. M. Fung, Hady W. Lauw, Longbing Cao.

Extended author information available on the last page of the article

1 Introduction

In today's fast-paced environment, learners, mid-career professionals, and job seekers increasingly face challenges in aligning their abilities with the evolving needs of high-demand occupations. This issue is intensified by technological advancements—including artificial intelligence (AI), machine learning (ML), and automation—that continually shape job markets. According to the World Economic Forum's Future of Jobs Report 2023, up to 85 million jobs could be lost by 2025, while 97 million new positions may arise, necessitating rapid upskilling in fields such as data analytics, cloud computing, and ethical governance in AI¹ (Gedrimiene et al., 2024). Recent research has highlighted the growing role of AI-driven educational technologies and learning analytics in supporting career development, skill acquisition, and personalized learning pathways for learners navigating dynamic job markets (Chen et al., 2020; Richter et al., 2019).

Recent innovations such as SkillGraph (Khan et al., 2025; Piro et al., 2024; Wang, 2025), and others (Shi et al., 2020) have addressed skill sequencing by leveraging knowledge graphs (Bellini et al., 2018) derived from resumes via OpenIE techniques, achieving 80.54% accuracy in skill recommendations across sectors. While this approach shows promise for profile-based guidance, it relies on direct skill mentions rather than capturing intent at query time (e.g., 'interpreting big data'). This limitation drives our focus toward intent extraction using transformers alongside prerequisite-aware Directed Acyclic Graph (DAG) optimization.

Traditional career guidance systems operate largely on rule-based algorithms or static keyword matches, which are insufficient to adequately capture these complexities (Career, 2025; Gowsika et al., 2025). Such systems compare user queries against pre-established job titles or skill inventories using exact or fuzzy string matches (e.g., with tools like Elasticsearch or TF-IDF vectorization), as described by Lops et al. (2010). However, this method often misses the semantic nuances of user intentions, leading to inappropriate recommendations and user dissatisfaction.

Career paths are intricate webs comprising prerequisites and inter-dependencies rather than mere lists of skills. For example, aspiring Machine Learning Engineers must first acquire proficiency in Python programming before advancing to TensorFlow, which itself requires knowledge in linear algebra and calculus. Research in competency-based education and intelligent tutoring systems has shown that effective learning pathways must explicitly model prerequisite relationships among skills to support learner progression and reduce cognitive overload (Esmail et al., 2025; Koedinger et al., 2013).

From an educational technology standpoint, this work seeks not only to match users with potential careers but also to orchestrate pedagogically coherent learning trajectories for self-directed learners. Many learners struggle to translate broad career aspirations into structured, prerequisite-aware learning plans, often resulting in fragmented knowledge acquisition and increased attrition rates (Layton et al., 2022). To address these challenges, this study introduces Cortex, an AI-driven career-aligned learning system designed to bridge the gap between career intent and actionable learning pathways. Figure 1 presents a high-level architecture diagram of the Cortex system. The system integrates semantic occupation matching, prerequisite-aware skill sequencing, and community-informed resource curation to generate structured learning sessions grounded in real-world skill taxonomies.

¹ <https://wallyboston.com/world-economic-forums-future-of-jobs-report-2023-published/>.

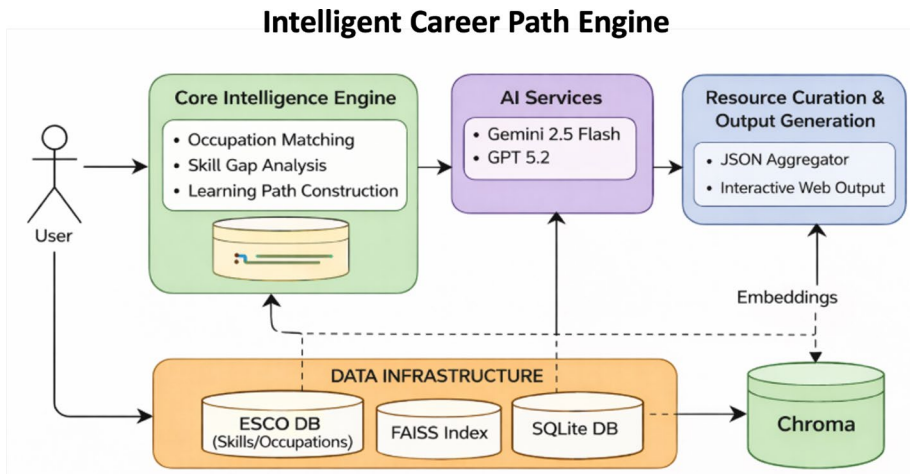


Fig. 1 High-level architecture

To systematically evaluate the effectiveness of the proposed Cortex system, the following research questions are investigated.

Research questions:

- **RQ1:** How effectively does a semantic embedding-based occupation matching approach (Sentence-BERT + FAISS) improve career recommendation accuracy compared to traditional keyword-based and TF-IDF methods?
- **RQ2:** To what extent can a RAG architecture constrain LLM generation to remain fully grounded in the ESCO skill taxonomy while preserving curriculum quality?
- **RQ3:** How does the integration of community feedback mechanisms influence the relevance, quality, and adaptability of generated learning paths?

1.1 Contributions

The key contributions of this work are summarised as follows.

- We leverage the prerequisite relations available in the ESCO taxonomy to generate personalized learning pathways, avoiding the need to construct prerequisite graphs from expert annotations or graph inference methods.
- We formulate career-goal matching as a dense semantic retrieval task using all-mpnet-base-v2 embeddings and FAISS over the ESCO occupation corpus, enabling occupation retrieval from free-text career goal descriptions.
- We develop a taxonomy-constrained retrieval-augmented generation framework that limits the generated skills to those defined in the ESCO taxonomy, ensuring consistency with the underlying skill database.
- We integrate semantic occupation retrieval, taxonomy-constrained skill recommendation, and prerequisite-guided learning pathway generation into a unified framework for AI-assisted career guidance.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature on intelligent tutoring systems, retrieval-augmented generation, and semantic occupation matching. Section 3 describes the dataset used. Section 4 describes the proposed Cortex system architecture and methodology, core intelligence engine, and AI services. Section 5 presents the experimental setup, evaluation metrics, and baseline methods used in the study. It also discusses the experimental results and provides a detailed analysis of system performance. Section 6 outlines ethical considerations and limitations, and Sect. 7 concludes the paper and outlines potential directions for future research.

2 Related Work

This paper builds upon recent advances in LLMs, Knowledge Graphs, and Recommender Systems in education. The following literature provides the theoretical foundation for the Cortex system.

“GenMentor,” an LLM-powered multi-agent framework for goal-oriented learning, was introduced by Wang et al. (2025). Their work demonstrated the efficacy of fine-tuning LLMs in goal-to-skill mappings and learner profiles. It addresses key limitations of Intelligent tutoring systems by enabling goal-oriented, personalized learning in professional contexts (Zerkouk et al., 2025). It maps learners’ objectives to specific skills using a fine-tuned LLM on a custom goal-to-skill dataset, identifies skill gaps, and generates efficient learning paths through an evolving optimization algorithm informed by dynamic learner profiles.

Tavakoli et al. (2022) proposed a hybrid model for personalized informal learning environments. Their research highlights the importance of combining AI automation with crowdsourced feedback loops. Contributors use AI-generated high-level goals, skills, topics, and content to collaboratively build and update dynamic curricula tailored to individual learners, which are integrated into a personalized online platform.

Lewis et al. (2020) formalized the concept of RAG, which combines parametric memory (pre-trained language models) with non-parametric memory (external databases). Unlike purely parametric seq2seq models limited by internalized knowledge, RAG employs a pre-trained neural retriever to dynamically fetch relevant passages with two variants: one conditioning on fixed passages across the sequence and another, allowing token-level passage variation, fine-tuned end-to-end for precise knowledge access and factual output (Gao et al., 2023).

Reimers and Gurevych (2019) developed Sentence-BERT, which allows the derivation of semantically meaningful sentence embeddings that can be compared using cosine similarity. Standard BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019) models incur massive computational overhead by processing sentence pairs jointly, requiring several hours to find the most similar pair among 10,000 sentences. However, SBERT reduces this considerably while preserving or even exceeding BERT’s accuracy.

Traditional sparse retrieval methods such as TF-IDF (Salton & Buckley, 1988) and BM25 (Robertson & Zaragoza, 2009) represent the predominant baselines for lexical occupation and job matching, relying on term frequency statistics to rank candidates. More recent dense retrieval approaches - exemplified by Dense Passage Retrieval (DPR) (Karpukhin et al., 2020) and general-purpose embedding models such as E5 (Wang et al., 2024) and BGE (Xiao et al., 2024) - replace sparse term overlap with learned semantic representations,

substantially outperforming sparse methods on semantic similarity tasks. Cortex builds on SBERT-based dense retrieval (Reimers & Gurevych, 2019) but further augments it with domain-specific, keyword-density, and career-transition boosting signals calibrated to the ESCO occupational vocabulary, a combination not evaluated in prior dense retrieval work.

Johnson et al. (2019) from Facebook AI Research introduced FAISS, a library for efficient similarity search of dense vectors. As datasets grow, linear search becomes computationally prohibitive. FAISS provides exact and approximate nearest-neighbour indexes that scale similarity search to millions of dense vectors while keeping query latency low.

Shi et al. (2020) proposed a multidimensional knowledge graph framework for learning path recommendation, encoding skill prerequisites, learner preferences, and content meta-data as a heterogeneous graph. Unlike Cortex, their system operates over a closed, platform-specific course catalog rather than an open taxonomy, and generates paths by graph traversal without LLM-based content generation. Khan et al. (2025) similarly apply knowledge graph reasoning to course recommendation but work within a fixed course catalog and do not generate learning sessions or enforce prerequisite ordering. Their system recommends existing courses from a fixed catalog; it does not generate learning sessions or impose prerequisite ordering on the generated path. Jiang et al. (2019) proposed goal-based course recommendation, mapping user goals to course sequences using intent modeling. Unlike Cortex, their system assumes a predefined course catalog and does not handle the semantic occupation-matching step (translating free-text career intent to a formal occupation) that Cortex addresses via SBERT + FAISS.

Table 1 provides a comparison of Cortex with other course recommendation systems on different factors.

The most closely related system is GenMentor, an LLM-powered multi-agent framework for goal-oriented learning (Wang et al., 2025). Like Cortex, it maps learner goals to skills, identifies gaps, and generates structured learning paths, but the two differ in architecture, grounding, and evaluation. GenMentor performs skill mapping with an LLM fine-tuned on a proprietary goal-to-skill dataset, whereas Cortex uses zero-shot semantic retrieval (Sen-

Table 1 Comparison of Cortex against closely related learning path and course recommendation systems

System	Approach	Open taxonomy	Prereq ordering	LLM generation	Hallucination control
Shi et al. (2020)	KG traversal	✗ (proprietary)	✓	✗	N/A
Khan et al. (2025)	GNN over KG	✗ (course catalog)	✗	✗	N/A
KnowLP (Cheng et al., 2026)	GraphRAG + dual KG	✗ (self-constructed)	✓	✓	Partial
CareerBERT (Decorte et al., 2023)	Resume + skill matching	✗ (resume data)	✗	✗	N/A
GenMentor (Wang et al., 2025)	Fine-tuned LLM	✗ (proprietary)	✓	✓	Partial
Cortex (this work)	SBERT+FAISS+RAG	✓ (ESCO)	✓	✓	DB-only constraint

Note: Comparison of closely related systems across five dimensions. Cortex is the only system that combines an open standardized taxonomy, prerequisite-aware ordering of learning path, LLM-based session generation, and hard taxonomy grounding within a single framework. DB-only refers to Database-only

tence-BERT embeddings with FAISS indexing) over the public ESCO taxonomy, requiring no fine-tuning and enabling direct reproducibility. For path scheduling, GenMentor applies an evolving optimisation algorithm driven by dynamic learner profiles; Cortex enforces pedagogically grounded ordering via ESCO prerequisite relations and topological sorting. For hallucination control, GenMentor integrates retrieved information during drafting but does not prevent out-of-taxonomy skills, while Cortex's DB-only RAG constraint blocks any skill absent from the ESCO database. A direct experimental comparison is therefore not feasible.

2.1 Research Gap

Despite significant progress in intelligent tutoring systems, semantic search, and AI-based recommendation frameworks, existing approaches often address these components in isolation. Many systems focus either on career recommendation based on user profiles or on learning path generation within predefined course catalogs, but few integrate semantic intent understanding with prerequisite-aware skill sequencing and curriculum structuring. Furthermore, several LLM-based approaches suffer from hallucination risks or lack mechanisms to ensure that generated learning paths remain grounded in validated skill taxonomies. These limitations highlight the need for a unified framework that combines semantic occupation matching, structured skill dependency modeling, and retrieval-augmented curriculum generation. To address these challenges, this work proposes Cortex, an AI-driven Career-aligned learning system that integrates semantic embeddings, knowledge graphs, and retrieval-augmented generation to produce coherent and scalable learning trajectories aligned with users' career goals.

3 Dataset

The core dataset is derived from the ESCO (European Skills, Competences, Qualifications, and Occupation) publicly available dataset². It is primarily an occupational taxonomy rather than a purpose-built pedagogical sequencing framework; its prerequisite relations reflect industry practice rather than optimally validated learning sequences. However, these relations provide a reproducible, expert-curated starting point for dependency ordering, which is an advantage over systems that construct prerequisite graphs from scratch or rely on LLM inference. This structured dataset ensures that the system is grounded in standardized professional terminology rather than in unstructured web data. It is an ontology-like knowledge base designed to support semantic interoperability between educational platforms, labor market systems, and HR tools.

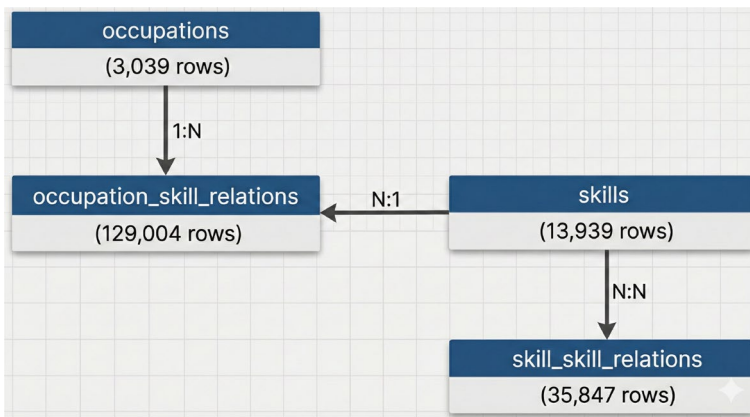
Table 2 provides highlights about the data. The data is normalized into a SQLite database³ (genmentor.db) to support relational queries. Figure 2 details the relationships between these entities.

²<https://esco.cc.europa.eu/en>

³<https://sqlite.org/>.

Table 2 Summary of dataset components used in the study

Dataset Component	Rows	Description
Occupations	3,039	Distinct occupational roles included in the knowledge base
Skills	13,950	Individual skills and competencies associated with occupations
Occupation-Skill Relations	129,004	Relationships linking occupations with the skills required to perform them
Skill-Skill Relations	5,818	Relationships indicating prerequisite or dependency links between skills (189 essential + 5,629 optional)

**Fig. 2** Data flow and component interaction

3.1 Pre-processing and Filtering

To ensure real-time performance, computationally intensive tasks are performed offline. A four-layered pre-processing pipeline was implemented. The details are summarized in Table 3.

Before creating the final learning paths, the following filters are implemented:

- Soft-skill filters: An internal list of generic traits (e.g., “communication,” “teamwork,” “adaptability”) is cross-referenced with the retrieved ESCO skill list. These soft skills are explicitly stripped/filtered out before the LLM generates the learning path.
- Low community vote filters: The system queries the community_votes metric in the database. If a mapped skill falls below a threshold (i.e., is heavily downvoted by previous users), it is actively filtered out and replaced during the learning path curation process.
- DB-only filters: The AI is instructed and systematically constrained via RAG to output only skills and modules that currently exist in the validated local SQLite database (ESCO dataset). The LLM is used to structure the plan, not to invent the skills.

Table 3 Pre-processing steps applied before similarity computation

Pre-processing Step	Description and Rationale	Parameter Values
Semantic embedding	Raw textual inputs are converted into dense vector representations using the Sentence Transformer model <i>all-mpnet-base-v2</i> . The model generates 768-dimensional embeddings, which are indexed using FAISS. Similarity between the user query and ESCO entries is then computed using cosine similarity to retrieve the most relevant matches.	NA
Domain-specific boosting	The system analyzes the user's career goal to identify explicit domain-related keywords. If a retrieved ESCO occupation belongs to the same domain, its FAISS similarity score is increased using a domain-based multiplier to prioritize domain-consistent results.	1.5x to 2.0x per match; cap 3x
Keyword density boosting	The algorithm measures the overlap between keywords in the user's expanded goal description and keywords present in ESCO occupation titles or descriptions. A higher overlap increases the ranking score through a keyword-based weighting mechanism.	1 + (overlap/union x 2.0): max 2x
Career transition boosting	When phrases such as "transition from X to Y" or "shift to" are detected, the system prioritizes the target role (Y) while still considering the source role (X). This strategy biases the retrieval process toward the destination occupation while identifying intermediary skills that facilitate the transition.	1.3x(beginner), 1.5x(marketing->data)

Note: Boost weights mentioned in the parameter value column were set heuristically based on pilot observations and were not tuned on a held-out development set; ablation results below quantify their individual contribution

4 Methodology

This section describes the methodology and implementation details of the proposed Cortex system using the dataset introduced in the previous section. Figure 3 presents the overall system architecture and illustrates the interaction between the major components of the framework. The Cortex system is organized into five primary modules: (a) the Core Intelligence Engine, (b) AI Services, (c) the Resource Curation Pipeline, (d) the Community Feedback Module, and (e) the Output Generation Layer. Together, these components enable semantic career matching, prerequisite-aware skill sequencing, and the generation of structured learning paths.

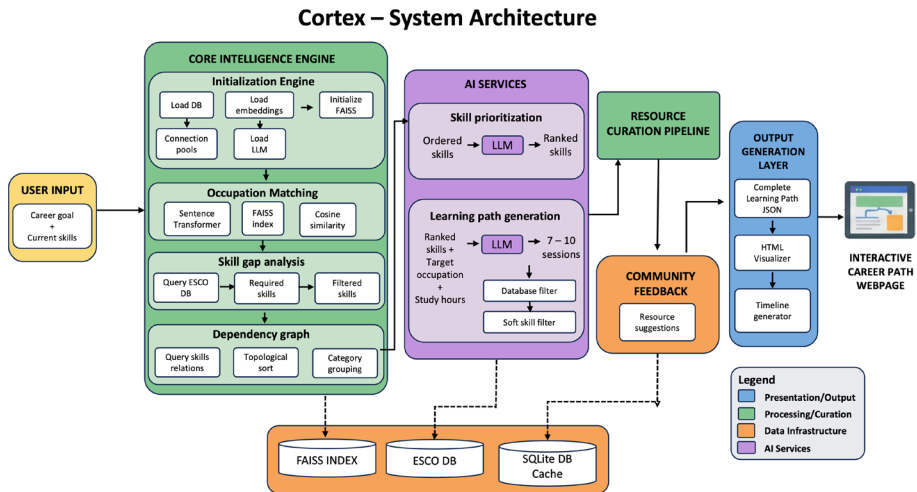


Fig. 3 System Architecture Diagram

Table 4 partitions the Cortex pipeline into three distinct processing layers. Rule-based components apply deterministic logic with no learned parameters. Retrieval-based components perform vector similarity search over the ESCO database. LLM-based components use generative models exclusively to structure and prioritise content. Critically, the LLM does not retrieve, create, or validate skills. Those functions are handled entirely by the rule-based and retrieval layers, ensuring that all skill content originates from the ESCO database.

4.1 Core Intelligence Engine

This is the system's entry point, where all the initialization happens. It serves as the foundational processing unit for preparing and analyzing data. Database creation, embedding generation, FAISS indexing, skill loading, and dependency graph construction are among the key activities performed during this phase.

- (a) **Initialization Engine:** This module handles startup procedures, including loading the database, initializing connection pools and embeddings, initializing the LLM, initializing FAISS, and the resource curator.
- (b) **Occupation Matching:** This module uses a Sentence Transformer (all-mpnet-base-v2)⁴ to create 768-dimensional embeddings. These are processed using a FAISS index with cosine similarity to match the user's input to specific occupations. When a user inputs a goal, the system tokenizes the input and pools the attention layers to create a normalized unit vector.

Because the ESCO occupation corpus is small (3,039 occupations), Cortex employs a FAISS IndexFlatL2 configuration, which performs exact nearest-neighbour search over unit-normalised embeddings; on normalised vectors, L2 ranking is equivalent to cosine-similarity ranking. Exact search at this corpus size answers queries in well under a millisecond, so

⁴<https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

Table 4 Details on the distinct layers of Cortex

Component	Layer	Input	Output
Goal embedding	Retrieval	Career goal text	768-dim dense vector
FAISS occupation search	Retrieval	Query vector	Top- <i>k</i> occupation candidates
Domain/keyword/transition boosting	Rule-based	FAISS scores + query tokens	Boosted similarity scores
Skill prioritization	LLM	Career goal text + candidate ESCO skills	Ranked skill list
Skill gap analysis	Rule-based	ESCO DB + declared user skills	Filtered skill list
Soft skill filter	Rule-based	Skill list	Soft skills removed
Beginner filter	Rule-based	Skill list + user skill count	Advanced topics removed
Dependency graph + topological sort	Rule-based	ESCO skill-skill relations (5,818 edges - 189 essential + 5,629 optional)	Prerequisite-ordered skill sequence
Session generation	LLM	Ranked skills + study hours	7 to 10 structured sessions (JSON)
DB-only filter (post-generation)	Rule-based	LLM JSON output + ESCO skills table	Validated session JSON
Resource curation	Rule-based + API	Validated skills	Learning resources per skill
Community feedback weighting	Rule-based	Vote scores + ESCO importance scores	Adjusted skill relevance

Note: The LLM is invoked only for Skill Prioritization and Session Generation. It receives ESCO-validated skill lists as context and does not retrieve, create, or validate skills. All skill content originates from the ESCO database; post-generation filtering enforces this constraint programmatically

no approximate indexing is required, and retrieval accuracy is unaffected by the index. For larger corpora ($\geq 10,000$ documents), the implementation automatically switches to an approximate IVF-Flat index (up to 100 K-means clusters, $n_{\text{probe}} = 10$) to trade a small recall loss for query speed, but that path is not exercised in the experiments reported here. The semantic occupation matching process implemented in the Cortex system is summarized in Algorithm 1.

Require: q : user career goal (free-text query)
Require: I : exact FAISS index (IndexFlatL2) over unit-normalised occupation embeddings $\{v_1, \dots, v_n\}$
Require: k : number of occupations to retrieve
Ensure: O : ranked list of relevant occupations

```

1:  $v_q \leftarrow \text{SENTENCETRANSFORMER.ENCODE}(q)$ 
2:  $v_q \leftarrow v_q / \|v_q\|_2$  {normalise to unit length}
3:  $N \leftarrow I.\text{SEARCH}(v_q, k)$  {indices of top- $k$  nearest neighbours}
4:  $O \leftarrow []$ 
5: for each occupation index  $i \in N$  do
6:    $\text{sim}_i \leftarrow v_q \cdot v_i$  {cosine similarity; equals L2 ranking on unit vectors}
7:    $O.\text{append}((occ_i, \text{sim}_i))$ 
8: end for
9: Sort  $O$  by  $\text{sim}$  in descending order
10:
11: return  $O$ 

```

Algorithm 1 Semantic Occupation Matching using Sentence Embeddings and FAISS

(c) **Skill Gap analysis:** This module queries the database to identify the required skills associated with the target occupation and produces a filtered set of skills that the user has not yet acquired. Multiple filtering mechanisms are applied to ensure that only relevant and actionable skills are considered during the generation of learning paths.

- **Soft Skill Filter:** Removes non-actionable skills like “critical thinking” or “communication” using a predefined set and partial matching. After the LLM generates the sessions, a filter removes any session whose skill list consists entirely of soft skills; whereas, sessions that pair soft skills with technical ones are retained. This is how interpersonal competencies can appear alongside technical skills.
- **Beginner Filter:** Removes advanced topics (e.g., “microservices orchestration”) if the user has fewer than 3 current skills. It identifies and excludes advanced skills when the user profile indicates limited prior knowledge (fewer than 3 declared current skills). A skill is classified as advanced using two criteria applied in combination: (a) Graph in-degree: Skills with three or more prerequisite dependencies in the ESCO skill-skill relations graph (i.e., in-degree ≥ 3 in the dependency DAG) are flagged as advanced, as they require substantial foundational knowledge before they can be meaningfully learned. (b) Keyword matching: Skills whose ESCO preferred label contains domain complexity markers (such as ‘orchestration’, ‘distributed’, ‘enterprise’, ‘architecture’, or ‘advanced’) are additionally flagged regardless of graph position. Skills excluded at this stage are not discarded permanently; they are stored as a deferred skill set and may be re-introduced in later sessions once foundational skills have been addressed.
- **Database-Only Filter:** Removes any hallucinations from the LLM output that do not match the ESCO database.

Require: *skill_list*: ESCO skills required for target occupation
Require: *user_skills*: skills declared by user
Require: *G*: dependency graph (nodes = skills, edges = prerequisites)
Require: *ADVANCED_KEYWORDS* = {'orchestration', 'distributed', 'enterprise', 'architecture', 'advanced'}
Ensure: *filtered_skills*, *deferred_skills*

```

1: is_beginner  $\leftarrow (|user\_skills| < 3)$ 
2: filtered_skills  $\leftarrow []$ 
3: deferred_skills  $\leftarrow []$ 
4: for each skill s in skill_list do
5:   if is_beginner then
6:     in_deg  $\leftarrow$  in-degree(s, G)
7:     has_keyword  $\leftarrow \exists kw \in ADVANCED\_KEYWORDS$  s.t. kw  $\subseteq$  s.label
8:     if in_deg  $\geq 3$  or has_keyword then
9:       deferred_skills.append(s)
10:    else
11:      filtered_skills.append(s)
12:    end if
13:  else
14:    filtered_skills.append(s)
15:  end if
16: end for
17:
18: return filtered_skills, deferred_skills

```

Algorithm 2 Beginner Filter

(d) **Dependency Graph:** To organize prerequisite relationships among skills, a dependency graph is constructed using the NetworkX library. Each skill is represented as a node, while prerequisite relationships form directed edges. A topological sorting algorithm is then applied to determine the appropriate learning order. The procedure used to generate the ordered learning path is summarized in Algorithm 3. All skill-to-skill dependency edges are derived exclusively from the ESCO skill_skill_relations table: 5,818 directed related-skill pairs labelled essential (189) or optional (5,629) over 5,650 skills, encoding ESCO relation_type = essential/optional. The LLM plays no role in generating or augmenting these edges, so the dependency structure contains no LLM-generated content. Cortex's RAG constraint ensures that only skills present in the ESCO database enter the graph, so every node and every edge is grounded in a validated external source. The full graph is acyclic once a small number of self-referential entries (present only among optional relations) are removed, and the essential-only subgraph (266 skills, 189 edges) is inherently acyclic. Coverage is sparse (41.7% of the 13,503 occupation-linked skills participate in at least one relation), falling to 1.9% for essential prerequisites, which in turn confirms ESCO's role as an occupational rather than a pedagogical resource and motivates the domain-expert edge augmentation identified as future work.

Require: Ranked skill set $S = \{s_1, \dots, s_n\}$; ESCO prerequisite relations R (`skill_skill_relations`); occupation relation map $\rho(s) \in \{\text{essential}, \text{optional}\}$; ESCO reuse level $\ell(s)$

Ensure: Ordered learning path L

- 1: Initialize empty directed graph G
- 2: **for** each skill $s_i \in S$ **do**
- 3: Add node s_i to G
- 4: **end for**
- 5: **for** each prerequisite relation $(s_i \rightarrow s_j) \in R$ with $s_i, s_j \in S$ **do**
- 6: Add directed edge $s_i \rightarrow s_j$ to G $\{s_i$ is prerequisite of $s_j\}$
- 7: **end for**
- 8: **if** G is acyclic **and** G has at least one edge **then**
- 9: $L \leftarrow \text{TOPOLOGICALSORT}(G)$
- 10: **else** {cycle detected, or no prerequisite edges among S }
- 11: $L \leftarrow$ skills of S sorted ascending by the composite key:
- 12: (i) $\text{rel}(s)$: essential $\rightarrow 0$, optional $\rightarrow 1$
- 13: (ii) $\text{reuse}(\ell(s))$: transversal $\rightarrow 0$, cross-sector $\rightarrow 1$, sector-specific $\rightarrow 2$, occupation-specific $\rightarrow 3$
- 14: (iii) global prerequisite in-degree $\deg_G^-(s)$
- 15: (iv) preferred label of s {deterministic tie-break}
- 16: **end if**
- 17: **return** ordered skill sequence L

Algorithm 3 Skill Dependency Graph Construction and Learning Path Ordering

4.2 AI Services

This layer acts as the brain of the system, transforming raw skill data into a structured educational experience. All the ordered skills from the dependency graph are selected and passed through the LLM to evaluate which skills are high-impact or act as prerequisite “blockers.” The model then outputs ranked skills, ensuring that the user addresses foundational concepts before progressing to more specialized ones.

Next, the ranked skills are combined with the target occupation and the user’s available study hours. The LLM uses this information to segment the skills into 7–10 logical learning sessions. The actual number of sessions is determined by skill volume and the user’s available study hours.

After session generation, the LLM output passes through a four-step validation pipeline. First, markdown fences are stripped and the response is parsed as JSON. If parsing fails, the system retries up to twice before falling back to a rule-based emergency session generator that groups the already-ordered skills into sessions by thematic category (Foundations, Programming, Data Tools, Machine Learning, Visualization, Cloud and Big Data, Specialized). This emergency path affects only session grouping. It is distinct from the learning-path ordering fallback of Algorithm 3, because skill order has already been fixed by the topological sort before session generation begins.

Second, a DB-only filter performs case-insensitive substring matching of each skill name against the ESCO skills table (13,950 entries), silently removing any skill not found. Third, a soft skill reintegration step appends a curated set of interpersonal skills (e.g., communication, teamwork) as a final supplementary session – these were deliberately excluded from LLM input to prevent disruption of prerequisite ordering, and are reintroduced here. Fourth, a resource availability check flags skills with no curated resource for the asynchronous API fallback pipeline. This pipeline ensures that all skills in the final output are ESCO-validated, prerequisite-ordered, and resource-backed.

4.3 Resource Curation and Community Feedback

Resources undergo a multi-tier curation process before user display, ensuring relevance, quality, and efficiency through dictionary lookups, fuzzy mapping, external API fallbacks, and asynchronous parallelism. This ensures that the learning path isn't just a list of titles, but a collection of high-quality, vetted materials.

(a) Resource Curation Pipeline

- **Curator Logic:** An automated system that matches skills to specific learning assets (e.g., videos, articles, and documentation). It checks a dictionary of **69 curated skill categories** and maps over **80 variations** (e.g., “web programming” → “JavaScript”).
- **External APIs (fallback):** When suitable resources are not found locally, the system queries external platforms such as *GitHub* and *DuckDuckGo* to retrieve relevant and up-to-date learning materials.
- **Asynchronous Processing:** Since API calls and data scraping can be time-consuming, these operations are executed asynchronously in the background to prevent delays in the web interface. The system uses *aihttp* and *asyncio* to fetch resources for multiple skills in parallel.
- **SQLite Cache:** Frequently accessed resources are stored in a SQLite cache, enabling faster retrieval for future requests related to similar career goals.

(b) **Community Feedback:** This module introduces a *human-in-the-loop* layer, allowing the curriculum to evolve based on user input. Depending on the feedback received, the suggested learning paths can be refined and updated. During cold-start, when no community votes exist for a skill, the system defaults to ESCO relation_type as the ranking signal: skills designated as essential in the ESCO occupation-skill relations are surfaced before those designated optional, providing a principled baseline ordering grounded in the ESCO taxonomy. As votes accumulate, community vote scores are incorporated directly into skill ranking alongside this ESCO-based ordering.

- **User Voting:** Allows the community to upvote high-quality resources and downvote outdated or less relevant ones.
- **Resource Suggestions:** Enables users to contribute new links or learning materials, which are then stored in a dedicated **SQLite database** for community-driven resource curation.

4.4 Output Generation Layer

This layer ensures that the generated learning path is not merely a list of titles but a collection of high-quality, vetted learning materials. The **JSON Aggregator** collects session data, quiz questions, and curated resources and combines them into a single JSON (JavaScript Object Notation) file. This file serves as the universal “source of truth” for the front-end interface.

Next, the **HTML Visualizer** converts the raw JSON data into a structured web format that can be rendered by a browser. Python libraries such as *NetworkX* and *Matplotlib* are

used to generate the visual learning map. NetworkX computes the structural relationships between skill nodes and their dependencies, while Matplotlib renders these relationships as a viewable graph.

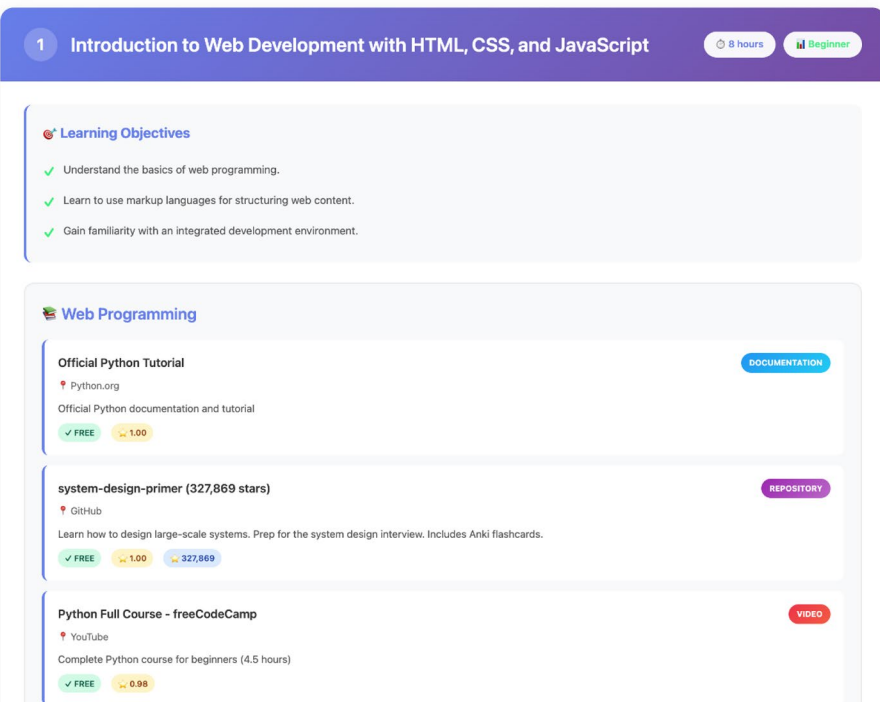
The final output is a dynamic dashboard where users can track their learning progress and access recommended resources. A sample output is illustrated in Fig. 4, which displays the user's goal, the list of recommended courses, the course type, and their corresponding resource links. A sample JSON output is provided in the Appendix (Sect. 8). All skill names in the output are validated against the ESCO skills table before the JSON is passed to the visualizer. Any skill name not matching an ESCO entry is removed by the DB-only filter.

4.5 Data Infrastructure

This module serves as the data foundation that supports the entire Cortex system.

- **ESCO Database:** A large, standardized database of European Skills, Competences, Qualifications and Occupations. It provides the “dictionary” the system uses to understand what a particular occupation (e.g., “Data Scientist” or “Plumber”) requires in terms of skills and competencies.
- **FAISS Indexing:** Developed by Meta, FAISS enables efficient vector search. It stores skill descriptions as mathematical vectors, allowing the system to identify the closest

Your Learning Path



The dashboard displays a learning path for "Introduction to Web Development with HTML, CSS, and JavaScript". It includes a section for "Learning Objectives" and a list of recommended resources under the "Web Programming" category.

1 Introduction to Web Development with HTML, CSS, and JavaScript 8 hours Beginner

Learning Objectives

- ✓ Understand the basics of web programming.
- ✓ Learn to use markup languages for structuring web content.
- ✓ Gain familiarity with an integrated development environment.

Web Programming

- Official Python Tutorial** (DOCUMENTATION)
 - Python.org
 - Official Python documentation and tutorial
 - ✓ FREE 1.00
- system-design-primer (327,869 stars)** (REPOSITORY)
 - GitHub
 - Learn how to design large-scale systems. Prep for the system design interview. Includes Anki flashcards.
 - ✓ FREE 1.00 327,869
- Python Full Course - freeCodeCamp** (VIDEO)
 - YouTube
 - Complete Python course for beginners (4.5 hours)
 - ✓ FREE 0.98

Fig. 4 Sample Learning Path Recommendation

matching occupation in a multi-dimensional vector space when a user enters a career goal.

- **SQLite and Cache:** Lightweight, disk-based databases are used to ensure efficient data access. The *cache* stores temporary data such as API results, while the *community database* stores persistent user interactions and feedback.

4.6 Large Language Model

In Cortex, the language model functions as a learning design assistant that organizes skills into structured, prerequisite-ordered learning sessions. These sessions are ordered according to prerequisite relationships rather than being arranged arbitrarily. Each session represents a coherent instructional unit with defined learning objectives, practice activities, and assessment criteria.

The model generates structured learning sessions from the ordered skill list, producing learning paths aligned with the user's specified career goal. To evaluate system performance, the framework was tested with multiple language models.

To ensure consistency and accuracy, the system enforces strict constraints so that only skills present in the database are used during generation. The target occupation and the list of skills stored in the database are provided within the system prompt. Cortex makes two distinct LLM calls per user request, at different pipeline stages. The first call operates during skill-gap identification, before the dependency graph is constructed. Given the user's career goal, the LLM scores candidate ESCO skills by relevance (critical, important, or useful), helping determine which skills are added to the skill-gap list. This call falls back to rule-based domain-keyword scoring if it fails or is unavailable. The second call operates after the dependency graph's topological sort has fixed the prerequisite order. It receives the already-ordered skill list and the user's declared weekly study hours, and groups the skills into 7 to 10 structured learning sessions. Critically, skill ordering itself is never LLM-determined. It is fixed entirely by the rule-based topological sort, or, when a cycle or missing edges prevent that, the taxonomy-grounded fallback of Sect. 4.1(d), before the second LLM call ever runs. This strict separation ensures that all skill names visible to either call are ESCO-validated. The post-generation DB-only filter enforces the same constraint regardless of which call structured the content.

Safety Settings: The model configuration includes safety filters that block harassment, hate speech, sexually explicit content, and dangerous content, ensuring that the generated output remains appropriate and reliable.

Table 5 presents the list of LLM parameters used for skill prioritization and learning content generation. The temperature parameter was varied between 0.2 and 0.4 to obtain optimal results. The *max_output_tokens* parameter was capped at 2048 to account for the API cost associated with token usage. The *top_p* value of 0.9 enables nucleus sampling, allowing the model to generate coherent responses while maintaining moderate linguistic diversity. Additionally, *top_k* is set to 40, restricting token selection to the most probable candidates at each generation step, thereby improving output focus and reducing noise.

Table 5 LLM hyperparameter configuration used in Cortex AI services

Parameter	Skill Prioritization	Session Generation	Rationale
Temperature	0.2	0.2	A low temperature encourages deterministic behavior, ensuring stable skill ranking and consistent learning session structures across runs.
Top- <i>p</i>	0.9	0.9	Nucleus sampling balances coherence and diversity by restricting token selection to the smallest probability mass covering 90% of likely tokens.
Top- <i>k</i>	40	40	Limits candidate tokens to the top 40 most probable options at each step, helping maintain focused and relevant outputs.
Max output tokens	2048	2048	Provides sufficient capacity for structured JSON responses and multi-step learning plans while controlling generation cost.

5 Performance Evaluation and Experimental Results

This section presents the results of the experimental evaluations conducted and summarizes the final outcomes.

5.1 Experimental Setup

Our evaluation comprises four complementary test sets, each targeting a different component and *unit of analysis*; their sizes therefore differ by design rather than by inconsistency:

- **25 occupation-retrieval queries** - natural language career goals, each with a single ground-truth ESCO occupation, used for the Top-*k*/MRR retrieval comparison (Table 9). Unit: a retrieval query.
- **20 functional evaluation scenarios** - end-to-end runs used to measure occupation-matching similarity scores (Fig. 5) and the internal consistency rate (Sect. 5.4). Unit: a full-system scenario.

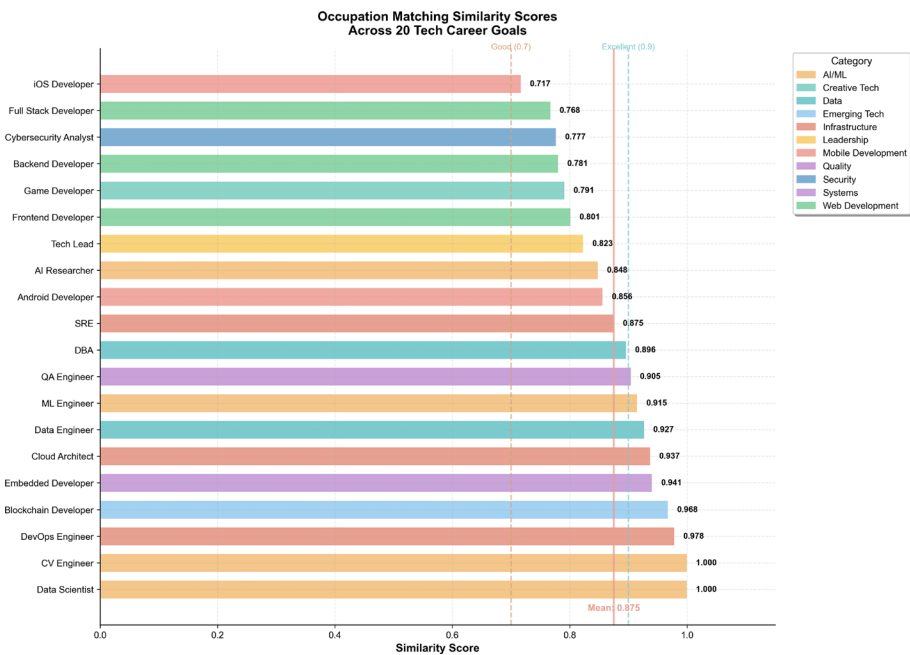


Fig. 5 Occupation matching similarity scores by test case, $N = 20$, focusing only on tech-related occupations

- **50 feature-level test executions** - fine-grained checks of individual system features (Fig. 6). Unit: a feature test.
- **8 generated learning paths** - complete paths assessed for prerequisite-ordering correctness (Table 7); the component ablations (Sect. 5.6) are likewise computed over generated learning paths. Unit: a generated path.

Because each set measures a distinct artifact - a retrieval query, an end-to-end scenario, a feature check, or a generated learning path - a single unified sample size is neither necessary nor meaningful; we report each experiment at the granularity appropriate to its metric. We acknowledge that the smaller sets (retrieval queries and generated paths) limit statistical power, which is why the retrieval comparison adopts a pre-registered single primary endpoint (Sec. 5.3) and why larger-scale evaluation is identified as future work. None of the test sets (there are four) was used during system development or parameter tuning.

Ground-truth labels were assigned independently by two authors (not independent third parties), with disagreements resolved by discussion against the ESCO preferred_label and description; as the labels are objective canonical ESCO occupations rather than subjective ratings, annotator bias is limited, and third-party evaluation is left to future work.

Table 6 shows the mapping of the obtained results to the corresponding research questions.

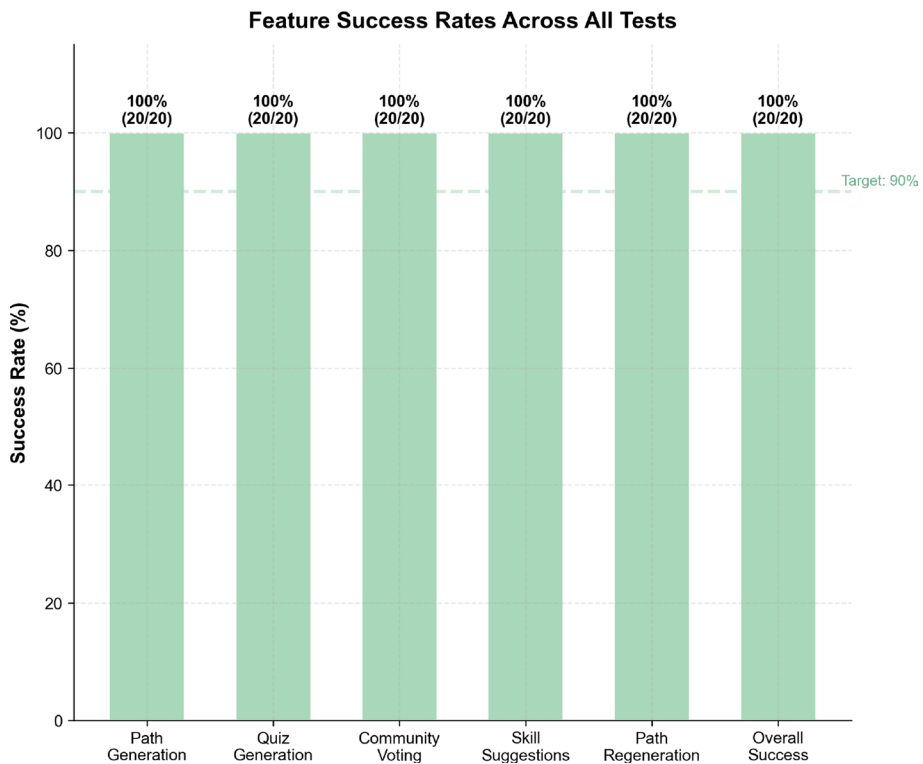


Fig. 6 Feature Success rates across all tests, $N = 50$, running the tests across all the defined test cases

Table 6 Mapping between research questions, experiments, and corresponding results

RQ	Experimental Evaluation	Corresponding Results
RQ1	Baseline model comparison	Table 9
RQ2	Constrained vs unconstrained RAG taxonomy-adherence and quality evaluation	Table 15 (Taxonomy adherence) and Table 7 (curriculum quality)
RQ3	Simulated community-feedback ablation	Table 13

5.2 Learning Path Quality Evaluation

We introduce a prerequisite violation rate metric to evaluate the pedagogical correctness of generated learning paths. A violation occurs when skill B depends on skill A in the ESCO skill relationship graph, yet B is assigned to an earlier session than A. We evaluated all 8 generated paths against the ESCO skill_relations table. The ESCO taxonomy contains 189 essential skill-skill edges across 13,950 skills; none connect skills that co-occur within a single generated path, reflecting a structural sparsity limitation of the taxonomy. Evaluating against optional dependency edges yields 5 checkable pairs across 5 of the 8 paths. Table 7 reports the results.

Table 7 Prerequisite Violation Rate Across Generated Learning Paths

Test Case	Skills	Sessions	Prereq Edges	Violations	Violation Rate
ML Engineer	7	3	0	0	—
DevOps Engineer	15	4	1	0	0%
Frontend Developer	13	3	1	0	0%
Backend Developer	15	3	0	0	—
Data Scientist	14	4	1	1	100%
Cloud Architect	15	5	1	0	0%
Cybersecurity Analyst	15	2	0	0	—
Full Stack Developer	13	3	1	0	0%
Aggregate	—	—	5	1	20%

Note: "-" denotes no checkable ESCO edges present in this path. The text in bold denotes aggregate values across the eight learning paths, not best or significant values.

Table 8 Per-occupation ordering outcomes ($N = 3,039$)

Ordering outcome (per occupation)	Full graph (ess. + opt.)	Essential-only
Topologically orderable (DAG)	2,502 (82.3%)	365 (12.0%)
Cycle→category fallback	5 (0.2%)	0 (0.0%)
No prerequisite edges→fallback	532 (17.5%)	2,674 (88.0%)
Edge coverage (of 13,503 occ.-linked skills)	41.7%	1.9%
Sub-graph edges (mean/median/max)	4.8/3/96	0.2/0/8

Note: Topological sorting orders most occupations under the full graph; the category fallback is triggered by genuine cycles in only 0.2% of cases, the rest driven by ESCO edge sparsity

Of the eight generated paths, five contain at least one checkable prerequisite edge, and four of these show no violations. The single exception is Data Scientist, where 'perform data mining' is placed in an earlier session (session 2) than its prerequisite 'data mining' (session 4), giving an aggregate violation rate of 20% (1 of 5 checkable pairs), down from 50% under the fixed-category fallback (that was first tested). This dependency is an optional ESCO skill-skill relation; because the ordering graph is constructed from essential relations only, optional dependencies are not enforced during topological sequencing, and the taxonomy-grounded fallback (essential-before-optional, then ESCO reuse_level) does not capture this fine-grained within-domain dependency. Incorporating optional and domain-expert prerequisite edges into the ordering graph is identified as future work (Sect. 7).

Table 8 reports per-occupation ordering outcomes: topological sorting governs 82.3% of occupations, and the fallback is invoked for fewer than one in five (almost entirely due to edge sparsity rather than cycles).

5.3 Baseline Comparison

Evaluation protocol: We evaluate occupation matching on 25 career-goal queries drawn from the ESCO technology cluster. Two authors independently assigned the ground-truth

ESCO occupation for each query and cross-checked their labels against the ESCO `preferred_label` and `description` fields. This query set was held out from system development and parameter tuning.

Methods compared: All methods index the same ESCO field, the concatenation of `preferred_label` and `description`, and none of the baselines receive the boosting used inside Cortex. We compare three lexical baselines and three dense-embedding baselines against two Cortex variants.

The lexical baselines are Keyword Matching, which ranks occupations by token overlap on `preferred_label`, TF-IDF, using `TfidfVectorizer` with `ngram_range=(1, 2)` and cosine similarity, and BM25, using `BM25Okapi` with $k_1 = 1.5$ and $b = 0.75$.

The dense-embedding baselines are all-MiniLM-L6-v2 (Wang et al., 2020), E5-base-v2 (Wang et al., 2024), and BGE-base-en-v1.5 (Xiao et al., 2024), each run without any boosting. These isolate how much of Cortex’s performance comes from dense embeddings in general, rather than from Cortex’s own boosting logic. The two variants of Cortex are Cortex (no boosts), which disables every boost heuristic, and Cortex (full), which uses `allmpnet-base-v2` (Reimers & Gurevych, 2019) with an exact FAISS flat-index L_2 (Johnson et al., 2019) and the complete boost pipeline.

Statistical analysis: We pre-registered a single confirmatory endpoint, per-query Reciprocal Rank, and compared Cortex (full) against each of the other seven methods using a paired Wilcoxon signed-rank test (Wilcoxon, 1945), with Holm-Bonferroni correction (Holm, 1979) applied across all seven comparisons. For each comparison we report Cohen’s d_z (Cohen, 1988; Lakens, 2013) and a 95% bootstrap confidence interval (Efron & Tibshirani, 1993) on the mean Reciprocal Rank difference. Per-class Top- k McNemar tests (McNemar, 1947) are exploratory only. Fixing one primary endpoint in advance keeps the analysis honest given the small sample size of $N = 25$.

Results: Results in Table 9 show two clear patterns. First, dense retrieval outperforms every lexical baseline by a wide margin. Second, the four dense configurations, MiniLM, E5, BGE, and Cortex (full), perform close to one another. E5-base-v2 reaches a nominally higher MRR than Cortex (full), 0.482 against 0.450, and MiniLM (0.476) and BGE (0.435) are similarly close. None of the three dense baselines differs significantly from Cortex (full) after Holm-Bonferroni correction ($p_{\text{Holm}} = 1.00$ for all three, with $d_z = -0.09$ for MiniLM,

Table 9 Baseline comparison of retrieval performance ($N = 25$ career-goal queries)

Method	Point estimates					vs. Cortex (full)		
	Top-1	Top-3	Top-5	Top-10	MRR	ΔMRR	d_z	p_{Holm}
Keyword Matching	16.0	28.0	28.0	40.0	0.232	+0.217	0.59	0.065
TF-IDF	20.0	28.0	40.0	48.0	0.275	+0.175	0.50	0.102
BM25	20.0	24.0	28.0	28.0	0.228	+0.222	0.66	0.019*
MiniLM-L6-v2 (no boosts)	36.0	56.0	68.0	72.0	0.476	−0.026	−0.09	1.00
E5-base-v2 (no boosts)	32.0	64.0	68.0	76.0	0.482	−0.033	−0.14	1.00
BGE-base-en-v1.5 (no boosts)	28.0	52.0	64.0	68.0	0.435	+0.015	0.11	1.00
Cortex (no boosts)	32.0	48.0	60.0	72.0	0.438	+0.012	0.06	1.00
Cortex (full)	32.0	56.0	64.0	72.0	0.450	n/a	n/a	n/a

Note: significant at $\alpha = 0.05$. Cortex (full system) is the reference arm. Top- k accuracy and MRR are point estimates. ΔMRR , d_z , and Holm-corrected p are the pre-registered confirmatory comparison of each baseline against Cortex (full system): per-query Reciprocal Rank, paired Wilcoxon signed-rank test, Holm-Bonferroni corrected across all seven comparisons

−0.14 for E5, and 0.11 for BGE). Among the lexical baselines, only BM25 reaches significance after correction ($p_{\text{Holm}} = 0.019$).

The three dense baselines use the same symmetric, SBERT-style cosine retrieval as Cortex. Because Cortex indexes with an exact FAISS flat- L_2 index, its rankings are identical to brute-force cosine similarity. FAISS changes retrieval latency here, not accuracy.

Retrieval is one component of Cortex. The system's contribution is the end-to-end, ESCO-grounded generation pipeline built around that retrieval step, not the retriever on its own. Using a compact, general-purpose encoder that performs comparably to larger dense baselines is consistent with that design goal.

Discussion: DPR's released checkpoints use an asymmetric question-and-context dual-encoder architecture, which differs from the symmetric encoders evaluated here. They are also trained on open-domain web question-answering data rather than occupational taxonomies. A zero-shot comparison against ESCO retrieval would therefore be mismatched on both architecture and domain. With only 25 queries, this study is also underpowered to separate closely matched dense methods from one another. We report MiniLM, E5, and BGE as comparable to Cortex (full) and leave a domain-adapted DPR comparison to future work.

An end-to-end comparison against existing learning-path generation systems is not yet possible. The closest published systems rely on proprietary datasets or closed course catalogs, and no standard public benchmark exists for career-aligned learning-path generation. The dense-retrieval baselines above are therefore the reproducible comparison available for the retrieval component. We intend the public release of Cortex's code, data, and evaluation pipeline to serve as such a benchmark for future work.

5.4 Quality Metrics and Calculation Methodology

The system's performance was evaluated using two key metrics. The *similarity score* measures how closely the returned results match the user-provided goal, while the *test pass rate* measures the overall success rate of the system. The calculation methodology and the results obtained for each metric are described below:

- (a) **Similarity Score:** This metric measures the semantic closeness between the user's natural language goal and the matched occupation.

Figure 5 illustrates the distribution of similarity scores across the evaluated cases. The results indicate that the Cortex system correctly matches the user's natural language goal to the appropriate occupation in approximately 88% of the cases.

- (b) **Internal Consistency Rate** (Target: 100%, Achieved: 100%) across 20 functional evaluation scenarios: This metric measures the success rate of the system across a comprehensive test suite consisting of 20 evaluation scenarios. It does not constitute an independent evaluation of pedagogical quality.

Criteria for Pass:

- **Keyword Match:** The matched occupation contains the expected keywords corresponding to the user's goal.

- **Skill Sufficiency:** The identified skill gap exceeds a minimum threshold (5 to 8 skills).
- **Path Validity:** The generated learning path contains valid learning sessions, appropriate durations, and no ESCO-unverified skills.

Figure 6 reports success rates across 50 individual feature-level test executions all of which passed.

The similarity score and internal consistency rate are system-internal diagnostics and not independent validity measures. External, ground-truth-based validation is instead provided by the occupation-retrieval evaluation against held-out ESCO labels with statistical significance testing (Table 9) and by the automatic prerequisite-violation metric computed against the ESCO skill graph (Table 7). Human and expert validation of pedagogical quality is not yet conducted and is identified as future work.

5.5 Complete System Flow Analysis

Table 10 illustrates the timing and logic of a typical user request (“I want to become a data scientist”). Table 11 summarizes the performance improvements achieved through the optimization techniques implemented in the system.

- **Occupation Matching:** By using a Sentence Transformer (*all-mpnet-base-v2*) to generate 768-dimensional embeddings, an FAISS index with cosine similarity is employed to match the user’s input with relevant occupations. Compared to a linear search approach, FAISS-based retrieval produced results approximately six times faster. The 0.02s reported in Table 10 refers to the FAISS nearest-neighbour lookup on the pre-built, in-memory index (warm path). This differs from the 0.86 s in Table 11, which reports the full occupation-matching subsystem (including embedding generation and index access) measured against the pre-optimization 5.2 s linear-search baseline.
- **Database Queries:** Query performance was improved through the use of connection pooling. By pre-creating and reusing database connections, the system avoids the $\approx$ 50 ms overhead associated with opening and closing connections for each query.
- **Resource Fetching:** Asynchronous processing was implemented using *aiohttp* and *asyncio* to fetch resources for multiple skills in parallel, thereby reducing latency in resource retrieval.
- **Overall System Performance:** The combination of the above optimizations resulted in a measurable improvement in the overall responsiveness and efficiency of the system.

5.6 Ablation Study

To isolate the contribution of each core component, we ablate them individually against the full system (Table 12). Disabling all three boosting heuristics (domain, keyword-density, and career-transition) changes retrieval MRR by only -0.012 (Holm-corrected $p = 0.88$; Table 12); because the combined effect is already negligible at $N = 25$, we do not decompose the boosts individually and leave a per-boost analysis to future work. Removing the dependency-graph ordering raises the prerequisite-violation rate to 54.5% across 22 checkable ESCO skill pairs, confirming that topological sequencing (not the LLM alone) drives

Table 10 Execution time analysis of the complete system pipeline

Processing Step	Time (s)	Description
Goal expansion and embedding	0.01	The Sentence Transformer model converts the user’s textual goal into a 768-dimensional embedding representation for semantic retrieval.
Occupation matching	0.02	FAISS retrieves the top candidate occupations, after which cosine similarity identifies the most relevant role (e.g., “Data Scientist”).
Skill gap identification	0.50	The ESCO knowledge base is queried to retrieve the required skills associated with the identified occupation.
LLM-based skill prioritization	3.50	A large language model ranks skills based on relevance, prerequisite relationships, and transition feasibility.
Skill gap computation	0.30	The difference between the user’s existing skills and the required skills for the target role is computed.
Dependency graph construction	0.10	NetworkX constructs a directed acyclic graph (DAG) of skill dependencies, and topological sorting determines the learning order.
Session generation	15.0	The LLM organizes prioritized skills into structured learning sessions while ensuring logical progression and prerequisite consistency.
Resource curation	4.50	Asynchronous resource retrieval fetches learning materials from curated databases and external APIs, followed by caching for future reuse.
Output generation	0.20	The final learning plan is assembled into structured JSON and rendered as HTML. The overall system response time is approximately 24 to 26 seconds, reduced from an earlier runtime of 45 seconds.

Table 11 Performance benchmarks before and after system optimizations

System Component	Before Optimization	After Optimization
Occupation matching	5.2 s (linear search)	0.86 s (FAISS-based retrieval)
Database queries	2.5 s	0.85 s (connection pooling)
Resource fetching	12 s (sequential requests)	3 s (asynchronous retrieval)
Overall system latency	≈45 s	≈24 to 26 s

Note: Component rows report isolated subsystem benchmarks measured independently. Overall system latency is the end-to-end wall-clock time of the full pipeline and corresponds to the step-by-step breakdown in Table 10

prerequisite correctness. Disabling the beginner filter increases the share of advanced skills surfaced from 0.0% to 5.8% (22 advanced skills removed from a 378-skill candidate pool over 20 cases). The soft-skill filter is a deterministic rule-based exclusion (soft skills are removed from the ordering stage and reintroduced as a supplementary session), so we report it qualitatively rather than as a metric ablation.

Table 12 Component ablation: effect of disabling each core component versus the full system

Component	Metric	Full → Disabled
Boosts (domain + keyword-density + transition)	Retrieval MRR	0.450 → 0.438 ($p = 0.88$, n.s.)
Dependency-graph ordering (DAG)	Prereq. violation rate [†]	0.0% → 54.5%
Beginner filter	Advanced-skill share	0.0% → 5.8%
Community feedback	Mean skill relevance	0.336 → 0.329 ($p = 0.32$, n.s.)

[†]Measured on the 20-case ablation run over 22 checkable ESCO skill pairs

Table 13 Effect of community feedback on skill recommendations

Measure	Value
Seeded votes (skills; simulated profiles)	444 (132; 303)
Recommended pairs involving voted skills	53
Voted skills added by feedback	27
Voted skills promoted/demoted/unchanged in rank	12/4/8
Voted skills removed by feedback	2
Mean skill relevance, feedback off→on	0.336 → 0.329
Scenarios improved/worsened/unchanged	6/5/9
Paired Wilcoxon signed-rank	$W = 21$, $p = 0.32$ (n.s.)
Bootstrap 95% CI of mean difference	[−0.015, −0.001]

Note: Each of the 20 evaluation scenarios was run twice. Once with community feedback enabled and once with it disabled, with all other settings unchanged. Skill relevance is computed using Eq. 1

Community feedback ablation. To evaluate the impact of the community feedback component, feedback was simulated by seeding upvotes for skills aligned with the target occupation and downvotes for low-priority skills. In total, 444 votes were seeded over 132 distinct skills from 303 simulated user profiles. We define the *skill relevance* of a skill s recommended for occupation o as

$$\text{rel}(s, o) = \cos(\mathbf{e}_s, \mathbf{e}_o), \quad (1)$$

where $\mathbf{e}_s$ and $\mathbf{e}_o$ are all-mpnet-base-v2 embeddings of the skill's ESCO label and description and of the occupation's ESCO description, respectively. The score ranges from −1 to 1 and higher values indicate closer alignment.

We re-ran the skill-recommendation stage for all 20 evaluation scenarios under two conditions on otherwise identical inputs. In the feedback-on condition the seeded votes were active, and in the feedback-off condition the votes table was empty. To isolate the vote effect and ensure exact reproducibility, occupation matches were held fixed at the values from the main evaluation run, since occupation matching does not read vote data. The LLM skill-priority assist was disabled in both conditions, so ranking used the deterministic keyword-priority fallback throughout. Under this setup the pipeline is fully deterministic, and a repeated run reproduces the outputs byte-for-byte.

At the mechanism level, feedback influences recommendations as designed (Table 13). Across the 20 scenarios, 53 recommended skill-occupation pairs involved voted skills. Feedback added 27 community-endorsed skills to the recommendations, such as Kubeflow and MLflow for the machine-learning scenario. It promoted 12 skills in rank, for example

PyTorch from rank 14 to 12 and TensorFlow from 13 to 11, each with 50 upvotes. It demoted 4 skills and removed 2, while 8 were unchanged.

Measured by Eq. 1, however, the per-scenario mean relevance of the recommended skill set did not improve with feedback. The mean was 0.336 with feedback off and 0.329 with feedback on, a mean paired difference of -0.007 , with 6 scenarios improved, 5 worsened, and 9 unchanged. A paired Wilcoxon signed-rank test over the 20 scenarios finds no significant effect ($W = 21$, $p = 0.32$) with zero differences dropped, $n = 11$, and $p = 0.77$ under the Pratt zero-handling procedure). The matched-pairs rank-biserial correlation is $r = 0.36$ toward negative shifts, and the bootstrap 95% confidence interval for the mean difference is $[-0.015, -0.001]$, computed with 10,000 resamples and seed 42.

The direction of this result is informative rather than anomalous. The negative shifts occur precisely in the scenarios where feedback *added* community-sourced skills, which score lower under Eq. 1 because they lie outside the ESCO taxonomy’s description vocabulary. Since the purpose of community feedback is to inject signal beyond taxonomy - embedding alignment, embedding-based relevance is a structurally conservative measure of its benefit and should not be read as the sole indicator of recommendation quality. Whether the community-driven additions and reorderings improve pedagogical usefulness is a question for human evaluation.

5.7 LLM Comparative Ablation

To assess the sensitivity of Cortex’s output quality to the choice of LLM generation backend, we evaluated the full Cortex pipeline with two different LLMs: Gemini 2.5 Flash⁵ and GPT 5.2⁶. Both models receive identical retrieved ESCO context, identical prompts, and identical post-generation filtering. So this comparison isolates the effect of the LLM component alone, holding all other pipeline components constant. This is not a comparison of Cortex against standalone LLMs; both configurations are full Cortex instances differing only in their generation backend. Table 14 shows the different parameters and how they differ between the two LLMs. Bold text denotes the highest-performing LLM for the corresponding evaluation metric.

GPT generated on average 32.8 skills per path versus 14.0 for *Gemini*, suggesting broader coverage at the cost of increased generation time; whether this translates to higher pedagogical quality requires human evaluation beyond the scope of this study. The identi-

Table 14 Comparative evaluation of LLM performance

Evaluation Metric	Gemini	GPT
Mean similarity score	0.8753	0.8753
Total evaluation time	21.3 min (faster)	37.0 min
Mean path generation time	9.1 s (faster)	30.5 s
Mean quiz generation time	24.4 s (faster)	29.2 s
Average number of skills generated	14.0	32.8 (higher)
Standard deviation of generated skills	2.3	13.5

Bold text denotes the highest-performing LLM for the corresponding evaluation metric

⁵<https://ai.google.dev/gemini-api/docs/models/gemini-2.5-flash>

⁶<https://developers.openai.com/api/docs/models/gpt-5.2>

cal mean similarity score (0.8753) for both LLMs indicates that occupation matching is performed upstream of the LLM, using the same SBERT+FAISS pipeline regardless of the LLM choice. This metric, therefore, measures retrieval quality, not generation quality, and confirms that both LLMs receive the same matched occupation context.

To quantify content breadth and depth objectively, we computed a category diversity score per path; the proportion of the eight Cortex skill categories covered by at least one skill. Both LLMs achieve identical mean diversity (0.36, covering 2.9 of 8 categories), confirming comparable breadth. The key difference is depth: GPT generates a mean of 4.92 skills per covered category versus 2.86 for Gemini, and produces more than twice as many skills overall (32.8 vs 14.0, $\sigma = 13.5$ vs 2.3). GPT leads across all categories except web development. These results support GPT's higher content density, though whether this translates into improved learning outcomes remains to be seen. Gemini's lower variance ($\sigma = 2.3$) produces more consistent, focused paths.

All metrics are computed automatically over the same 20 test occupations; no human annotation was involved. The unconstrained system uses the same LLM (Gemini 2.5 Flash) and decoding parameters (temperature 0.2, top-p 0.9), without the ESCO skill list in the prompt and without post-generation filtering. Generated skills are matched against the 13,950-entry ESCO skills table (99,242 preferred and alternative labels) at three tiers: exact string match (case-folded), normalized match (parentheticals and punctuation removed), and semantic match (all-mpnet-base-v2 cosine against the nearest ESCO label, reported at thresholds 0.80–0.90). Content completeness is the proportion of the target occupation's top-10 essential ESCO skills (by relevance score) covered by the generated path, computed identically for both systems.

The results of this analysis are presented in Table 15. The findings show that the constrained RAG system produces output fully grounded in the ESCO taxonomy by construction, whereas only 3.1% of skills generated by the unconstrained system match ESCO labels exactly, rising to 19.1% under semantic matching.

5.8 Outlier Analysis

Table 16 presents the identified outlier cases along with the reasons for their deviation from the remaining test cases. The analysis indicates that most discrepancies stem from **limitations in the ESCO occupational taxonomy rather than shortcomings in the proposed semantic matching approach**.

Table 15 Taxonomy adherence and essential-skill coverage

Metric (N = 20 occupations)	Constrained RAG	Unconstrained
ESCO taxonomy adherence, exact (†)	100% (by construction)	3.1% (12/382)
— after label normalization	100%	6.0% (23/382)
— semantically groundable (cos $\geq$ 0.80)	100%	19.1% (73/382)
Content completeness (top-10 essential skills), normalized (†)	41.0%	0.0%
— semantic match (cos $\geq$ 0.85)	44.5%	1.5%

Note: 382 skills generated by the unconstrained system. Metric definitions and the evaluation protocol are given in Sect. 5.7

Table 16 Analysis of representative outlier cases in occupation matching

User Goal	Similarity	Observed Issue	Primary Cause
iOS developer	0.717 (lowest)	Matched to “Embedded Systems Developer” instead of a mobile-application specific occupation	The ESCO taxonomy lacks a precise occupation category for iOS or mobile application developers.
Full Stack Developer	0.768	Matched to the broader occupation “Web Developer”	The ESCO occupational taxonomy does not explicitly represent the full-stack developer role.
Cyber Security Analyst	0.777	Matched to “Computer Scientist” rather than a specialized security role	Absence of a dedicated cybersecurity occupation in the ESCO dataset, combined with limitations of semantic embedding similarity.
AI Researcher	0.848	Generated only one learning session despite high similarity	Limited coverage of recent AI research skills in the current ESCO knowledge base.

Table 17 extends the outlier analysis to long-tail and emerging occupations. We identify three outcome tiers based on ESCO coverage. For occupations with adequate skill coverage (≥ 20 linked skills), Cortex achieves similarity scores comparable to popular roles (≥ 0.82). For occupations with sparse coverage (< 20 skills), correct occupation matching is maintained, but generated learning paths are shorter and less granular. For modern roles with zero ESCO coverage (notably cloud DevOps engineer and cloud identity manager), which appear in the ESCO occupation taxonomy but lack linked skills, Cortex falls back to the nearest broader occupation (e.g., ICT system administrator). These cases represent a known limitation of ESCO v1.2 rather than a system failure: as ESCO is updated, coverage will improve automatically since Cortex’s retrieval is database-driven. To mitigate this gap in the short term, zero-coverage occupations are handled by the taxonomy-grounded fallback of Algorithm 3, which orders the skills of the nearest broader occupation by relation type, reuse level, and prerequisite in-degree to generate a plausible generic path.

6 Ethical Considerations and Limitations

The following are a few ethical considerations addressed in this work, along with a few limitations.

- (a) **Dataset bias and coverage limitations:** While ESCO provides a well-structured, standardized taxonomy, it is inherently Europe-centric and may under-represent emerging roles, informal skill pathways, and region-specific occupations. Consequently,

Table 17 Long-Tail Occupation Evaluation Performance on Data-Sparse and Emerging Roles

ID	Query(Abbreviated)	Expected	Matched	Similarity	Skills	Tier
LT01	I want to manage cloud identity and access systems	Cloud Identity Manager	Cloud Identity Manager	0.73	24	Good
LT02	I want to advise organisations on sustainable ICT practices	Green ICT consultant	Green ICT consultant	0.720	27	Good
LT03	I want to use sensor data and statistics to predict equipment failures	Green ICT consultant	Green ICT consultant	0.585	25	Good
LT04	I want to manage data communications systems for aviation operations	Aviation data communications manager	Aviation data communications manager	0.708	15	Partial
LT05	I want to build and deploy cloud-native software applications	cloud software developer	cloud software developer	0.708	27	Good
LT06	I want to implement DevOps practices for cloud infrastructure	cloud DevOps engineer	cloud DevOps engineer	0.702	30	Good
LT07	I want to design digital games for gambling and casino platforms	gambling games designer	gambling games developer	0.712	20	Fallback
LT08	I want to manage game development projects and teams	games development manager	games development manager	0.593	17	Partial

Note: Coverage tiers: *Good* = correct match, ≥ 20 ESCO skills; *Partial* = correct match, < 20 skills; *Fallback* = broader occupation matched due to zero ESCO skill coverage. Similarity is cosine similarity between query and top-1 occupation embedding

recommendations generated may be less accurate for users operating in non-European labor markets or rapidly evolving industries.

- (b) **Algorithmic bias in career recommendations:** Semantic similarity-based occupation matching may unintentionally favor well-documented or high-frequency roles in the dataset, potentially narrowing the diversity of career suggestions. To mitigate this risk, the system emphasizes explainable similarity scores and exposes multiple ranked occupation candidates rather than a single deterministic recommendation.
- (c) **LLM Hallucination and content safety:** LLMs are known to hallucinate facts or introduce non-existent concepts when unconstrained. Cortex mitigates this risk through a strict RAG framework that limits all skill and curriculum generation to entities retrieved from the ESCO dataset. Post-generation filtering further ensures that any content not grounded in the databases is discarded, significantly reducing the risk of misinformation.
- (d) **Privacy and user data handling:** Cortex operates with minimal user data requirements and does not collect personally identifiable information beyond voluntary skill inputs and feedback signals. All processing is performed on anonymized input, and no long-term learner profiling is retained. The design minimizes privacy risks while still allowing for personalized guidance.
- (e) **Technical and methodological limitations:** Despite its strengths, Cortex has several limitations. First, learning paths are generated in a largely static manner and do not yet adapt dynamically to the learner's progress over time. Second, evaluation focuses primarily on system-level metrics rather than long-term educational outcomes such as

skill retention or career success. Finally, certain thresholds and filtering heuristics (e.g., beginner skill limits, domain boost factors) are empirically chosen and may require further optimization through large-scale user studies.

7 Conclusion and Future Work

The experimental results confirm that the Cortex system effectively addresses the research questions raised in this study. **RQ1** is addressed by the results, which indicate that semantic embedding-based occupation matching achieves higher recommendation accuracy than BM25 ($p_{\text{Holm}} = 0.019$). The improvement over keyword matching and TF-IDF is a positive but non-significant trend at $N = 25$. **RQ2** is addressed by the findings, which suggest that the hard, database-constrained RAG architecture guarantees taxonomy-grounded (ESCO-valid) skill generation by construction, whereas unconstrained generation is largely unmapable to the taxonomy and its essential-skill coverage is near zero; whether unconstrained outputs contain semantically false content remains unquantified. **RQ3** is addressed by the experimental results, which show that a community-feedback signal, evaluated through a simulated voting proxy applied in a single iteration, influences skill recommendations as designed by adding, promoting, and demoting community-endorsed skills. Its effect on embedding-based skill relevance is statistically non-significant, and whether these changes improve pedagogical usefulness is a question for human evaluation. Longitudinal validation with real users over multiple sessions is reserved for future work.

Integration of the ESCO data set ensures that these pathways are grounded in industry standards, while the hard RAG constraint guarantees that generated content remains grounded in the ESCO taxonomy, addressing a key reliability risk associated with Generative AI. The technical implementation highlights the importance of hybrid architectures—combining the creativity of LLMs with the logical rigor of Knowledge Graphs and the speed of vector databases like FAISS.

By integrating semantic intent modeling with pedagogically grounded skill sequencing, Cortex contributes to the growing body of learning technologies that aim to scaffold self-directed learning in complex and evolving professional domains. This approach bridges the gap between static job descriptions and dynamic, personalized learning, ultimately empowering users to navigate their professional development with clarity and confidence. It strongly supports coherent learning trajectories while reducing fragmentation in self-directed learning. Full ESCO adherence is enforced by post-generation filtering rather than observed LLM behavior. Semantic errors are plausible but pedagogically incorrect skills or sequences remain an open evaluation challenge.

Future work may focus on the following areas:

- RL-based curriculum adaptation: Future Cortex iterations will incorporate Reinforcement Learning from Human Feedback (RLHF) to enable dynamic curriculum adaptation. By modeling the learner's progress as a Markov Decision Process, the system can continuously optimize skill sequencing based on user performance and feedback signals.

- Longitudinal learning outcome studies: Future work will focus on validating educational effectiveness through controlled user studies. By conducting longitudinal experiments with pre- and post-assessments, the impact of the system on learning outcomes, retention, and career readiness can be empirically measured. This includes multi-session validation of the community-feedback loop with real users.
- Expanded test scope: Future work can involve covering broader domains and more test scenarios for testing.
- Prerequisite graph enrichment: The ESCO-derived dependency graph is sparse, with 41.7% of occupation-linked skills participating in at least one relation and only 1.9% in essential prerequisites. Future work will augment the ordering graph with optional dependencies and domain-expert-curated prerequisite edges to capture fine-grained within-domain sequencing.
- Expanded retrieval benchmarking: The current evaluation ($N = 25$) is underpowered to separate closely matched dense retrievers. Future work will include a domain-adapted DPR baseline, a larger occupation benchmark, and a per-heuristic ablation of the domain, keyword-density, and career-transition boosts.
- Human and expert validation: Pedagogical quality has so far been assessed only through automatic, system-level metrics. Future work will add expert review of generated learning paths, third-party annotation of ground-truth labels, and human evaluation of whether broader skill coverage (e.g., GPT versus Gemini) translates into higher pedagogical quality.
- Threshold and heuristic optimization: Empirically chosen parameters (beginner skill limits, domain boost factors, and filtering thresholds) will be tuned through large-scale user studies rather than fixed heuristics.

Appendix

A. Sample datasets used - Fig. 7 provides details of some data records present in the `generator.db`, which is constructed from the ESCO data set.

conceptType	occconceptUri	iscoGroup	preferredLabel	altLabels
Occupation	http://data.europa.eu/esc	2654	technical director	technical and operations director
Occupation	http://data.europa.eu/esc	8121	metal drawing machine operator	metal drawing machine technician
Occupation	http://data.europa.eu/esc	7543	precision device inspector	inspector of precision instruments
Occupation	http://data.europa.eu/esc	3155	air traffic safety technician	air traffic safety electronics hardware
Occupation	http://data.europa.eu/esc	2431	hospitality revenue manager	hospitality revenues manager
Occupation	http://data.europa.eu/esc	3212	medical laboratory assistant	medical laboratory research
Occupation	http://data.europa.eu/esc	3116	asphalt laboratory technician	road surface technician
Occupation	http://data.europa.eu/esc	5312	primary school teaching assistant	primary school teaching aide
Occupation	http://data.europa.eu/esc	2264	physiotherapist	neuromusculoskeletal
Occupation	http://data.europa.eu/esc	2310	performing arts theatre instructor	acting school teacher
Occupation	http://data.europa.eu/esc	8160	hydrogenation machine operator	hydrogenation machine operative
Occupation	http://data.europa.eu/esc	8160	pasta operator	pasta products operator
Occupation	http://data.europa.eu/esc	1420	tobacco shop manager	smokers products shop manager
Occupation	http://data.europa.eu/esc	5249	rental service representative in oth	rental sales desk clerk in other

Fig. 7 Snapshot of data from the ESCO data set

B. Sample test cases - Fig. 8 provides details for 10 sample test cases and occupation details.

ID	Career Goal	Matched Occupation	Similarity	Skills	Sessions	Path Gen (s)	Quiz Gen (s)
1	ML Engineer	Computer Vision Engineer	0.915	7	2	20.74	24.35
2	DevOps Engineer	Cloud DevOps Engineer	0.978	15	4	11.07	23.43
3	Frontend Developer	Web Developer	0.801	13	3	8.07	26.65
4	Backend Developer	User Interface Developer	0.781	15	3	8.61	29.35
5	Data Scientist	Data Scientist	1	14	4	7.62	18.99
6	Cloud Architect	Cloud Engineer	0.937	15	5	8.35	28.14
7	Cybersecurity Analyst	Computer Scientist	0.777	15	2	8.74	27.15
8	Full Stack Developer	Web Developer	0.768	13	3	7.89	23.02
9	Android Developer	Mobile App Developer	0.856	15	3	8.32	25.62
10	iOS Developer	Embedded Systems Dev	0.717	15	3	8.57	26.33

Fig. 8 Ten sample test cases

C. Sample screenshots of generated learning paths - Figs. 9 and 10 provide snapshots of a sample learning path generated in Cortex.

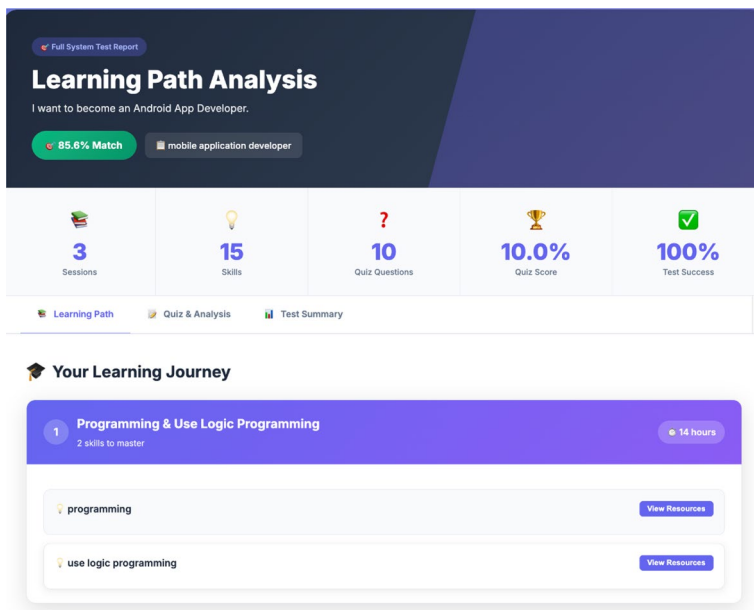


Fig. 9 Snapshot of Learning Path

2

Android Sdk, Android (mobile Operating Systems) & More (10 Skills)

10 skills to master

20 hours

Android Sdk

Hide Resources

Android Developers Offici...
Google • Documentation

Android Basics with Com...
Google • Course

Android Development Full...
freeCodeCamp • Video

Android Codelabs
Google • Practice

Android (mobile operating systems)

Hide Resources

Android Developers Offici...
Google • Documentation

Android Basics with Com...
Google • Course

Android Development Full...
freeCodeCamp • Video

Android Codelabs
Google • Practice

JavaScript

View Resources

mobile operating systems

View Resources

mobile device software frameworks

View Resources

Gradle

View Resources

Firebase

View Resources

Room Database

View Resources

Fig. 10 Snapshot of Learning Path

D. LLM prompts used for skill prioritisation and session generation are shown in Fig. 11.

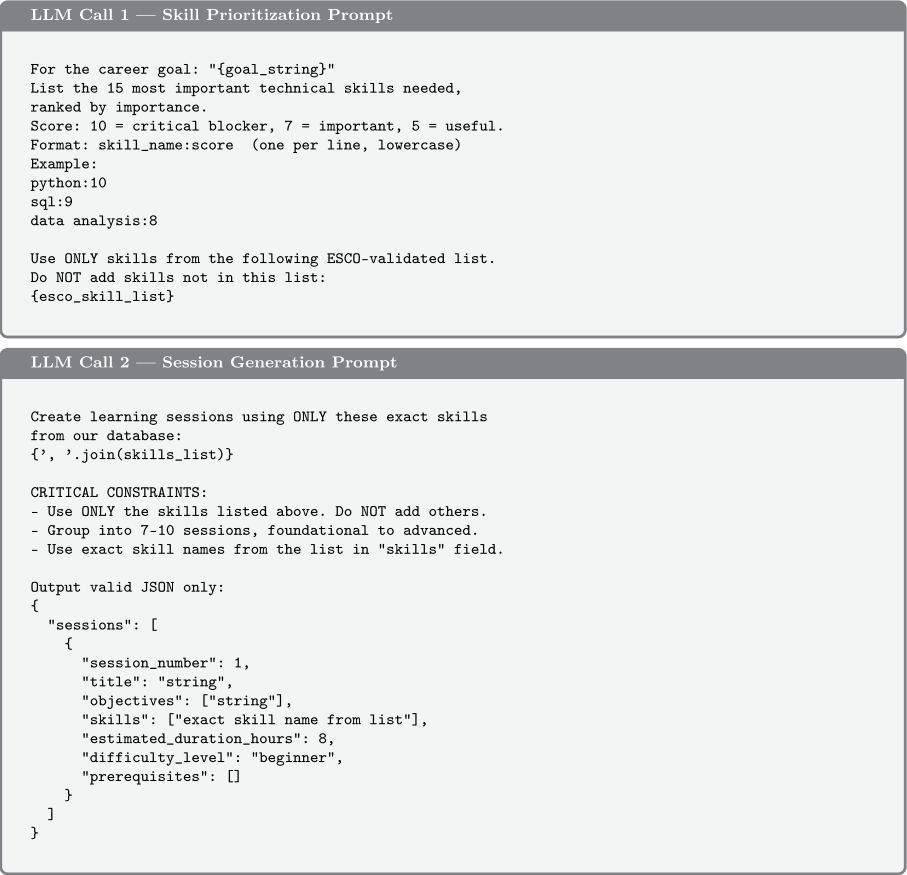


Fig. 11 Full prompt templates for the two LLM calls in Cortex. Runtime variables shown in {braces}. Post-generation DB-only filtering enforces the ESCO constraint programmatically regardless of prompt compliance

E. Sample system prompt is shown in Fig. 12.

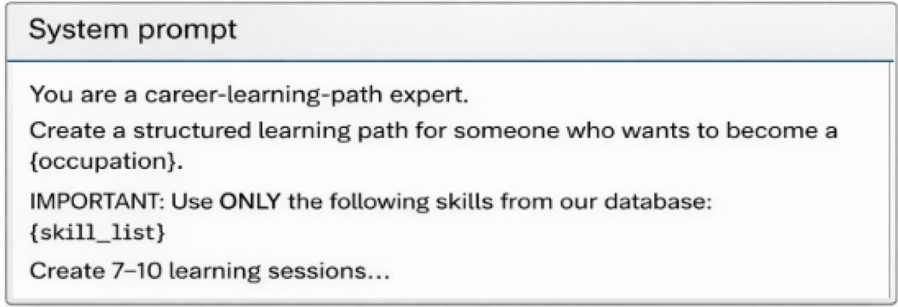


Fig. 12 System Prompt Setting

F. Sample JSON output is shown in Fig. 13.

```
json
{
  "occupation": "Data Scientist",
  "total_sessions": 4,
  "sessions": [
    {
      "session_number": 1,
      "title": "Foundations of Programming and Data Handling",
      "difficulty_level": "beginner",
      "estimated_duration_hours": 12,
      "objectives": [
        "Write Python scripts for data manipulation",
        "Query relational databases using SQL",
        "Apply basic statistical analysis to datasets"
      ],
      "skills": [
        "Python (programming language)",
        "SQL",
        "data analysis"
      ],
      "prerequisites": []
    },
    {
      "session_number": 2,
      "title": "Machine Learning Fundamentals",
      "difficulty_level": "intermediate",
      "estimated_duration_hours": 16,
      "objectives": [
        "Understand supervised and unsupervised learning",
        "Implement models using scikit-learn"
      ],
      "skills": [
        "machine learning",
        "statistical modelling",
        "data visualisation"
      ],
      "prerequisites": ["Python (programming language)", "data analysis"]
    }
  ]
}
```

Fig. 13 Sample Output JSON

G. Sample test scenarios are provided in Table 18.

Table 18 Sample of 8 executed end-to-end evaluation scenarios in Cortex

#	Exact Test Input (goal + current skills)	Category	Matched ESCO Occupation	Skills	Sess.	Path (s)
01	I want to become a Data Scientist. (current skills: python, statistics)	AI/ML	data scientist	23	5	131.0
02	I want to become a Machine Learning Engineer. (current skills: python, linear algebra, pandas)	AI/ML	computer vision engineer	18	10	105.9
03	I want to become a DevOps Engineer. (current skills: linux, bash scripting, git)	Infrastructure	cloud DevOps engineer	15	5	158.1
04	I want to become a Frontend Developer. (current skills: html, css, javascript)	Web Development	web developer	32	10	95.8
05	I want to become a Backend Developer. (current skills: python, sql)	Web Development	user interface developer	27	15	117.9
06	I want to become a Cloud Architect. (current skills: networking, linux, virtualization)	Infrastructure	cloud engineer	25	8	148.8
07	I want to become a Full Stack Developer. (current skills: javascript, html, css, node.js)	Web Development	web developer	24	8	174.1
08	I want to become an Android App Developer. (current skills: java, object-oriented programming)	Mobile Development	mobile application developer	26	15	148.0

*Each input supplies the learner's prior current skills. The verbatim input, matched ESCO occupation and recorded metrics are shown. Skills = skills in the generated path (after excluding recognised current skills); Sess. = learning sessions; Path (s) = path-generation time in seconds

H. Example out-of-taxonomy skills generated by the unconstrained system are shown in Table 19.

Table 19 Example out-of-taxonomy skills generated by the unconstrained system (ML Engineer test case)

	Generated skill (unconstrained LLM)	Nearest ESCO label	Cosine
	Data Structures and Algorithms	algorithms	0.66
	Machine Learning Fundamentals (Supervised, Unsupervised, Reinforcement Learning)	perform machine learning	0.61
	Deep Learning Frameworks (TensorFlow, PyTorch)	TensorFlow	0.64
	Model Training and Evaluation	evaluate the training	0.65
	Model Deployment (e.g., REST APIs, Docker, Kubernetes)	Kubernetes	0.61
	Cloud Platforms (AWS, Azure, GCP)	cloud technologies	0.71
	Big Data Technologies (Spark, Hadoop)	Hadoop	0.65
	MLOps Principles and Practices	ML development principles	0.67
	Version Control (Git)	software tools for version control	0.78

*Skills that could not be grounded in the ESCO taxonomy at any matching tier (exact, normalized, or semantic at cosine ≥ 0.80). The outputs are plausible curriculum phrasing but do not correspond to ESCO entries, illustrating the grounding gap that the DB-only constraint closes

Author Contributions V.R. and M.V. conducted all the experiments under the supervision of S.K. V.R., M.V., and S.K. wrote, reviewed, and finalized the manuscript.

Data Availability The datasets and source code used in this study are publicly available to support reproducibility of the proposed system. The implementation of the Cortex framework, along with the experimental scripts and evaluation pipeline, can be accessed at: <https://github.com/Manasvi-Vedanta/Cortex>. The occupational skill data used in this study are derived from the publicly available ESCO (European Skills, Competences, Qualifications and Occupations) database.

Declarations

Conflict of interest The authors declare no competing interests.

References

- Bellini, V., Schiavone, A., Di Noia, T., Ragone, A., & Di Sciascio, E. (2018). Knowledge-aware autoencoders for explainable recommender systems. In *Proceedings of the 3rd workshop on deep learning for recommender systems*, pp. 24–31. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3270323.3270327>
- Career guidance system for school children. (2025). *International Research Journal of Engineering and Technology (IRJET)*, vol. 12, no. 5. [Online]. Available: <https://www.irjet.net/archives/V12/i5/IRJET-V121577.pdf>
- Chen, L., Chen, P., & Lin, Z. (2020). Artificial intelligence in education: A review. *IEEE access*, 8, 75264–75278. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/9069875>
- Cheng, X., Zhang, Z., Wang, J., Fang, L., He, C., Guan, Q., Pan, S., & Luo, W. (2026). Graphrag-induced dual knowledge structure graphs for personalized learning path recommendation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40, 14610–14620.
- Cohen, J. (1988). *Statistical Power Analysis for the Behavioral Sciences* (2nd ed). Lawrence Erlbaum Associates.
- Decorte, J.-J., Van Haute, J., Deleu, J., Develder, C., & Demeester, T. (2023). Career path prediction using resume representation learning and skill-based matching, arXiv e-prints, pp. arXiv–2310,
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: Human language technologies, volume 1 (long and short papers)*, pp. 4171–4186.
- Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall/CRC.
- Esmail, A. A., Elsayed, M. A., Abdallah, S. M., Mohamed, A. E., Fouad, K. M., & Essa, E. (2025). Enhancing career path guidance through comprehensive student performance analysis. In *International Conference on Intelligent Systems, Blockchain, and Communication Technologies*. Springer, pp. 134–146. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-032-09562-6_10
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., Wang H. (2023). Retrieval-augmented generation for large language models: A survey, arXiv preprint [arXiv:2312.10997](https://arxiv.org/abs/2312.10997), vol. 2, p. 32.
- Gedrimiene, E., Celik, I., Kaasila, A., Mäkitalo, K., & Muukkonen, H. (2024). Artificial intelligence (ai)-enhanced learning analytics (la) for supporting career decisions: Advantages and challenges from user perspective. *Education and Information Technologies*, 29, 297–322. [Online]. Available: <https://link.springer.com/article/10.1007/s10639-023-12277-4>
- Gowsika, S., Aparna, S., Kiruthika, V., & Lavanya, K. (2025). Personalized career guidance system for students, *International Journal of Advanced Research in Science, Communication and Technology (IJARSCT)*. [Online]. Available: <https://ijarsct.co.in/Paper23904.pdf>
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2), 65–70.
- Jiang, W., Pardos, Z. A., & Wei, Q. (2019). Goal-based course recommendation. In *Proceedings of the 9th international conference on learning analytics & knowledge*, pp. 36–45.
- Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with gpus, *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547 [Online]. Available: <https://ieeexplore.ieee.org/document/8733051>
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & tau Yih, W. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781.

- Khan, M. A. Z., Luo, D., & Polyzou, A. (2025). Reasoning with knowledge graphs for trustworthy course recommendation. In *2025 IEEE 12th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE, pp. 1–10. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/11248000/?casa_token=WEEog9tQsY8AAAAA:YeuYSx7x7HUyWG3qibXXDt2_T4DujS9OIDqsc6LnOAMSZy5hTDi--HijBjHdy_PBNSe_Ol17Bdzg3g
- Koedinger, K. R., Brunskill, E., Baker, R. S., McLaughlin, E. A., & Stamper, J. (2013). New potentials for data-driven intelligent tutoring system development and optimization. *AI Magazine*, 34(3), 27–41.
- Lakens, D. (2013). Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and anovas. *Frontiers in Psychology*, 4, 863.
- Layton, R. L., Solberg, V. S. H., Jahangir, A. E., Hall, J. D., Ponder, C. A., Micoli, K. J., & Vanderford, N. L. (2022). Career planning courses increase career readiness of graduate and postdoctoral trainees. *F1000Research*, 9, 1230.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, [Online]. Available: <https://dl.acm.org/doi/abs/10.5555/3495724.3496517>
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). Roberta: A robustly optimized bert pretraining approach. arXiv preprint [arXiv:1907.11692](https://arxiv.org/abs/1907.11692)
- Lops, P., Gemmis, M. D., & Semeraro, G. (2010). Content-based recommender systems: State of the art and trends. *Recommender systems handbook* (pp. 73–105)
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.
- Piro, L., Bianchi, T., Alessandrelli, L., Chizzola, A., Casiraghi, D., Sancassani, S., & Gatti, N. (2024). Mylearningtalk: An llm-based intelligent tutoring system, in *International Conference on Web Engineering*. Springer, pp. 428–431
- Reimers, N., & Gurevych, I. (2019). Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992.
- Richter, O. Z., Juarros, V. I. M., Bond, M., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education: where are the educators? *International Journal of Educational Technology in Higher Education*, 16, 6. [Online]. Available: <https://dialnet.unirioja.es/servlet/articulo?codigo=7288782>
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: Bm25 and beyond, in *Foundations and Trends in Information Retrieval*. Now Publishers, 3(4), 333–389.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
- Shi, D., Wang, T., Xing, H., & Xu, H. (2020). A learning path recommendation model based on a multidimensional knowledge graph framework for e-learning. *Knowledge-Based Systems*, 195, Article 105618.
- Tavakoli, M., Faraji, A., Molavi, M., Mol, S. T. & Kismihók, G. (2022). Hybrid human-ai curriculum development for personalised informal learning environments. In *LAK22: 12th International learning analytics and knowledge conference*, pp. 563–569. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3506860.3506917>
- Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F. (2024). Multilingual e5 text embeddings: A technical report, arXiv e-prints, pp. arXiv–2402.
- Wang, M. (2025). Skillgraph-align: A skill graph enhanced job recommendation method for university graduates—a decade-long validation on real-world data. In *2025 IEEE International Conference on Big Data (BigData)*. IEEE, pp. 3548–3554.
- Wang, T., Zhan, Y., Lian, J., Hu, Z., Yuan, N. J., Zhang, Q., Xie, X., & Xiong, H. (2025). Llm-powered multi-agent framework for goal-oriented learning in intelligent tutoring system. In *Companion Proceedings of the ACM on Web Conference 2025*, pp. 510–519. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3701716.3715244>
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Proc. NeurIPS*
- Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6), 80–83.
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). Bge m3-embedding: Multi-lingual, multi-functional, multi-granularity text embeddings through self-knowledge distillation. [arXiv:2402.03216](https://arxiv.org/abs/2402.03216).
- Zerkouk, M., Mihoubi, M., & Chikhaoui, B. (2025). A comprehensive review of ai-based intelligent tutoring systems: applications and challenges. arXiv preprint [arXiv:2507.18882](https://arxiv.org/abs/2507.18882).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Vatsala Ramachandran¹  · Manasvi Vedanta¹  · Sonia Khetarpaul¹ 

✉ Sonia Khetarpaul
sonia.khetarpaul@snu.edu.in

Vatsala Ramachandran
vr337@snu.edu.in

Manasvi Vedanta
mv901@snu.edu.in

¹ Department of Computer Science and Engineering, Shiv Nadar Institution of Eminence, Greater Noida, , Uttar Pradesh, , India